

Visualização em Tempo Real

Apresentação Trabalho Prático

Ivan Ribeiro, Francisco Ferreira e Júlio Pinto

Simulador de partículas determinístico com GPU

- Rust
- WGPU

Segurança, performance, controlo sobre a pipeline de renderização, APIs modernas

- Grande escala
- Determinismo
- Colisões

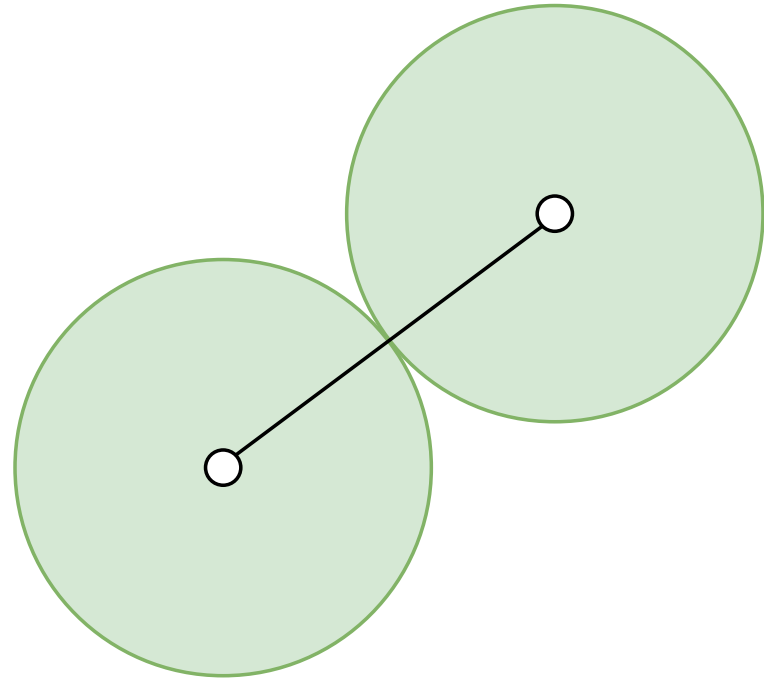
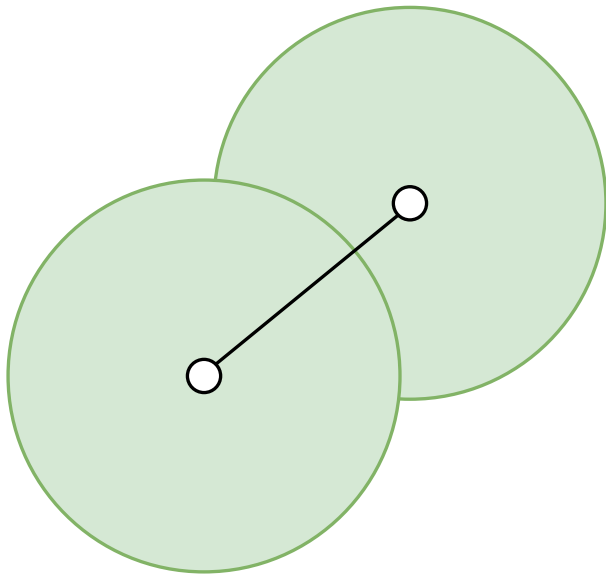
Que equações usar para mover as partículas? E para as colidir?

- **Basic Verlet Integration**
- Velocity Verlet
- Euler integration
- Leapfrog Integration
- Position Verlet
- Runge-Kutta Methods (e.g., RK4)
- Backward Euler

```
struct ParticlePhysics {  
    pos: Vec2,  
    old_pos: Vec2,  
    accel: Vec2,  
}
```

Em cada passo, dado um Δt :

```
let vel = particle.pos - particle.old_pos;  
particle.old_pos = particle.pos;  
particle.pos += vel + (particle.accel * DELTA_SQUARED);  
particle.accel = Vec2::ZERO;
```



Otimizações

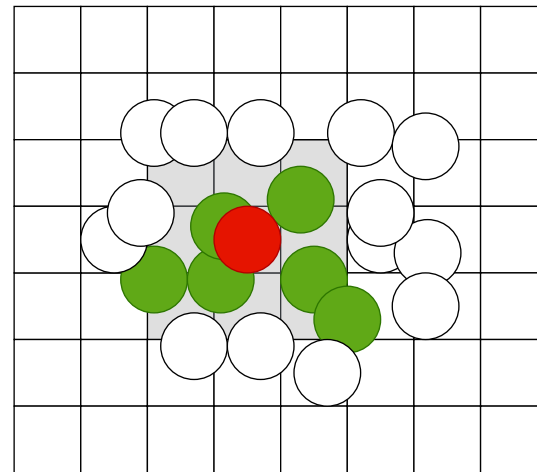
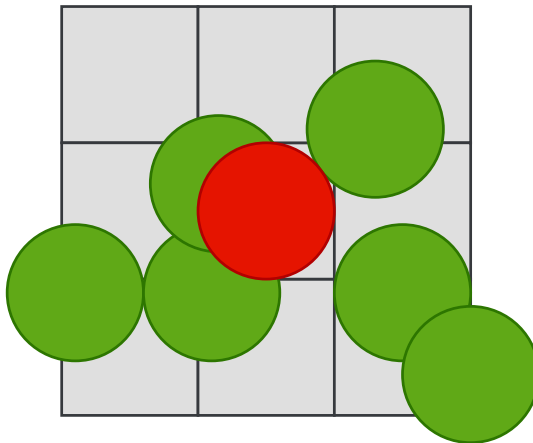
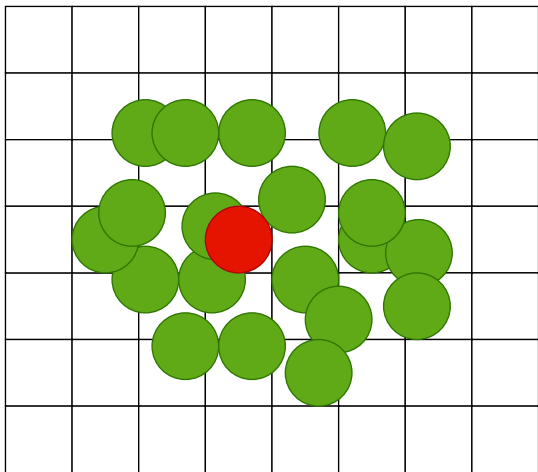
Atual:

- Comparar todas as partículas com todas as outras (N^2)
- Não paralelizável
- **10k** partículas: **4FPS**

Objetivo:

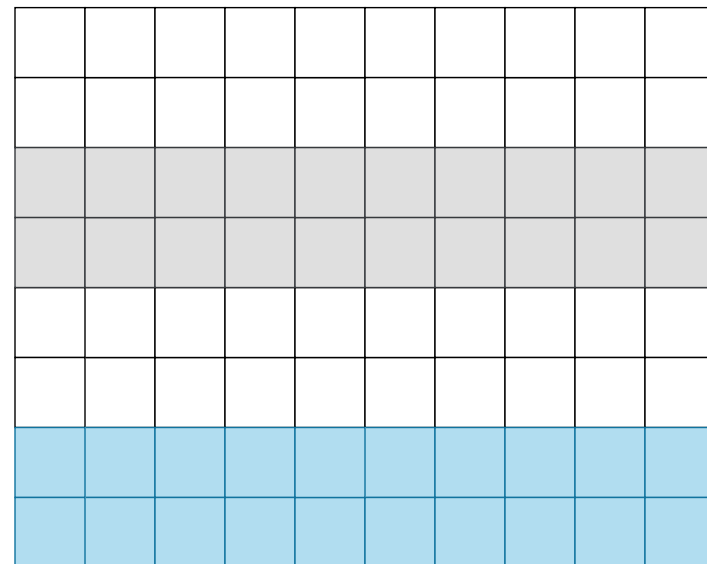
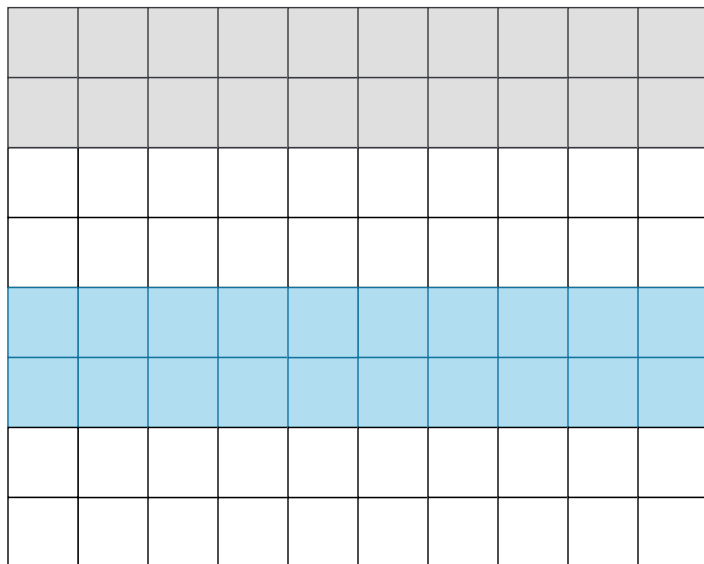
- **100k** partículas, **60FPS**

- Dividir o espaço em grelha
- Comparar células adjacentes



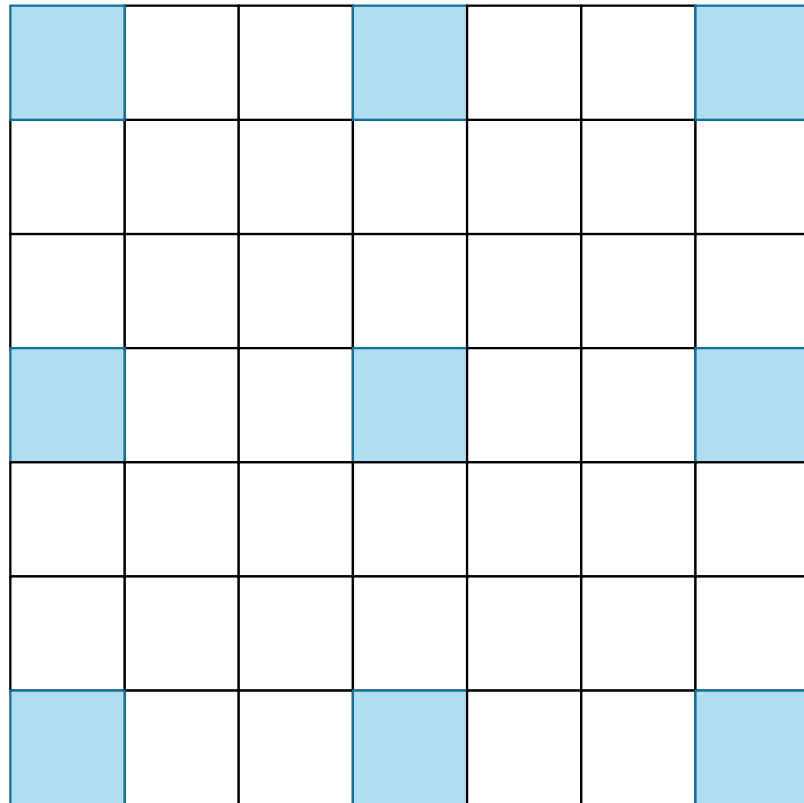
- **10k** partículas: **300FPS**

- Cada thread processa colisões numa parte da grelha
- Deixa de ser determinístico: são necessários 2 passes
- Colocar partículas na grelha tem de continuar a ser sequencial

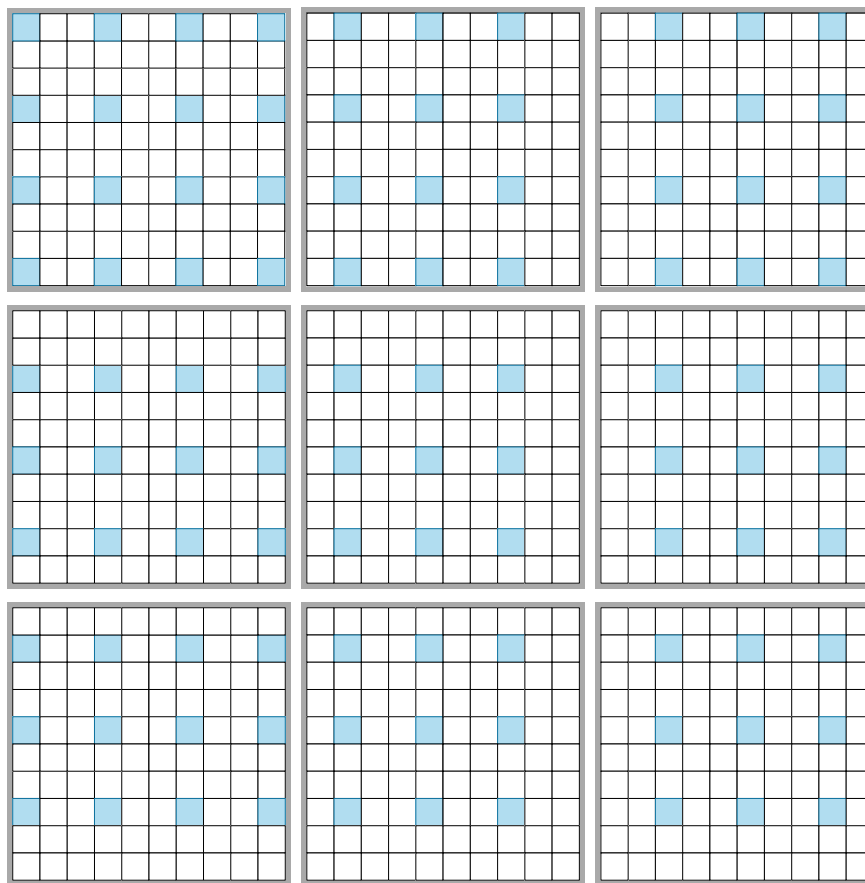


- **50k** partículas: **50FPS**

- Não podemos usar a mesma estrutura do multithreading
- Como manter determinismo?



- São necessários 9 passes
- **100k** partículas: **120FPS**



Renderização de imagens

- Determinismo permite que partículas acabem todas na mesma posição
- Se atribuirmos cores às partículas, elas poderão ser usadas para mostrar imagens



Spawners

- Quisemos permitir animações complexas
- Spawners são responsáveis por criar partículas com uma certa posição, direção, etc

```
struct Spawner {  
    start_frame: u64,  
    end_frame: u64,  
    spawn_every_n: u64,  
    spawner_type: SpawnerType, // dir, pos, etc  
}
```

